Learn Home Automation with Arduino

Lesson 5: Wireless Communication with Bluetooth

Wireless Communication with Bluetooth – Study Notes

Key Concepts

1. **HC 05 Bluetooth module** – a widely used, inexpensive serial (UART) Bluetooth /2.0 slave device for Arduino projects.

2. **Serial communication (UART)** – the HC 05 talks to the Arduino via TX/RX pins at a configurable baud rate (default 9600 /bps).

3. **Bluetooth pairing & SPP (Serial Port Profile)** – how a smartphone or PC discovers, pairs, and creates a virtual COM port with the HC 05.

4. **AT command mode** – special command set used to configure the HC 05 (change name, password, role, baud rate, etc.).

5. **Mobile app !' Arduino command flow** – typical steps: pair !' open a Bluetooth terminal / custom app !' send ASCII/byte commands !' Arduino reads and acts.

Summary

The HC 05 module lets an Arduino exchange data wirelessly with a smartphone, tablet, or PC using classic Bluetooth (not BLE). In a typical "phone to Arduino" setup, the phone runs a simple Bluetooth terminal app (or a custom app built with MIT App Inventor, Arduino Bluetooth Control, etc.). After pairing, the phone's app opens an SPP connection that appears as a virtual serial port on the HC 05. The Arduino reads incoming bytes from its hardware serial (or a SoftwareSerial port) and interprets them as commands (e.g., turn an LED on/off, move a motor, request sensor data).

Before using the module, you may need to place it in AT command mode to set the device name, PIN, or baud rate to match your sketch. Once configured, the module works in "data mode" where everything sent from the phone is streamed directly to the Arduino's serial buffer. By structuring the data (e.g., using newline terminated strings or fixed length packets) you can reliably parse commands and send responses back to the phone.

Important Points

- **Wiring**

- HC 05 VCC !' 5 /V (or 3.3 /V for some modules)
- GND !' GND
- TXD !' Arduino **RX** (use a voltage divider if the HC 05 is 3.3 /V tolerant)
- RXD !' Arduino **TX** (HC 05 tolerates 5 /V logic, but a 1 /k:'ó" ö³© divider is recommended)

- **Power up sequence**

- LED blinking fast ("H2 /s) !' module in **AT mode** (if KEY pin held HIGH).
- LED steady blinking (~2 /s) !' **Data mode** (ready for normal communication).

• **AT command basics**

- Enter AT mode: hold KEY pin HIGH, power up.
- Default baud: 38400 /bps in AT mode, 9600 /bps in data mode.
- Example: `AT+NAME=MyBT` !’ rename the module.

• **Serial handling in Arduino**

```cpp
#include <SoftwareSerial.h>
SoftwareSerial BT(10, 11); // RX, TX
void setup() {
  Serial.begin(9600);      // PC monitor
  BT.begin(9600);          // HC  05
}
void loop() {
  if (BT.available()) {
    char c = BT.read();          // read one byte
    // simple command parsing
    if (c == '1') digitalWrite(LED_BUILTIN, HIGH);
    else if (c == '0') digitalWrite(LED_BUILTIN, LOW);
  }
}
```

• **Command formatting**

- Use a terminator (`\n` or `\r`) so the Arduino knows when a full command has arrived.
- For multiple values, separate with commas: `"LED,1\n"` !’ split with `strtok()`.

• **Safety**

- Do **not** connect the HC 05 VCC to the Arduino’s 5 /V pin if the module is 3.3 /V only.
- Keep the KEY pin un connected (or low) during normal operation to avoid accidental AT mode entry.

Examples

1. Turn an LED on/off from a phone

Hardware: HC 05 wired to Arduino UNO (SoftwareSerial on pins /10/11). LED on pin /7.

Arduino sketch (see code block above, with LED pin added).

Phone app: Use “Serial Bluetooth Terminal” (Android) or “Bluetooth Serial Controller” (iOS).

- Pair with HC 05 (default PIN `1234`).
- Send `1` !’ LED turns **ON**.
- Send `0` !’ LED turns **OFF**.

Explanation: The phone sends a single ASCII character. The Arduino reads it, checks the value, and writes to the LED pin accordingly.

2. Request sensor data and display on phone

Hardware: HC 05 + Arduino + temperature sensor (LM35 on A0).

Arduino sketch

```cpp
#include <SoftwareSerial.h>
SoftwareSerial BT(10, 11);
void setup() {
  Serial.begin(9600);
  BT.begin(9600);
}
void loop() {
  if (BT.available()) {
    String cmd = BT.readStringUntil('\n');
    cmd.trim();
    if (cmd == "TEMP") {
      int val = analogRead(A0);
      float tempC = val * (5.0 / 1023.0) * 100.0;
      BT.println(String(tempC, 1));   // send back temperature
    }
  }
}
```

Phone app: Same terminal app, type `TEMP` and press send. The Arduino replies with the temperature (e.g., `23.4`).

Explanation: The command `TEMP` triggers the Arduino to read the analog sensor, convert to Celsius, and send the result back over the same Bluetooth serial link.

Quick Review

1. **What pins on the Arduino are typically used for HC 05 communication, and why might you use SoftwareSerial?**
2. **How do you put the HC 05 into AT command mode, and what is the default AT mode baud rate?**
3. **Why is it a good idea to terminate commands with a newline character?**
4. **Describe the basic steps a phone app follows to control an Arduino via Bluetooth.**
5. **What safety precaution should you take when connecting the HC 05's RX pin to a 5 /V Arduino?**

Further Reading

- **Bluetooth Classic vs. BLE** – when to choose HC 05 vs. HM 10.
- **Advanced AT commands** – setting role as master, configuring UART flow control.
- **Using the Arduino “Serial” monitor together with Bluetooth** – multiplexing data streams.
- **Creating a custom Android app** with MIT App Inventor or Android Studio for richer UI.
- **Security considerations** – changing default PIN, disabling discoverability, and encrypting data.